

```

public String getErrFileName() {
    return this.errFileName;
}

public void setErrFileName(String errFileName) {
    this.errFileName = errFileName;
}

public String getErrMethodName() {
    return this.errMethodName;
}

public void setErrMethodName(String errMethodName) {
    this.errMethodName = errMethodName;
}

public int getPgmlineNo() {
    return this.pgmlineNo;
}

public void setPgmlineNo(int pgmlineNo) {
    this.pgmlineNo = pgmlineNo;
}

public String[] getMsgValue() {
    return this.msgValue;
}

public void setMsgValue(String[] msgValue) {
    this.msgValue = msgValue;
}

public String getErrorPageUrl() {
    return this.errorPageUrl;
}

public void setErrorPageUrl(String errorPageUrl) {
    this.errorPageUrl = errorPageUrl;
}

@Override
public String toString() {
    return "NhbaseException [statusCode=" + statusCode + ", stderrMsgContents="
        + stderrMsgContents + ", addMsgContents=" + addMsgContents + ", serviceUrl="
        + serviceUrl + ", errFileName=" + errFileName + ", errClassName=" + errClassName + ", errMethodName="
        + errMethodName + ", pgmlineNo=" + pgmlineNo + ", errMsgType=" + errMsgType + ", msgValue="
        + Arrays.toString(msgValue) + ", errorPageUrl=" + errorPageUrl + "];"
}

}

package nhnits.fw.commons.exception;

import java.util.Arrays;

```

```

import org.apache.logging.Log4j.ThreadContext;
import nhis.fw.commons.exception.NhBaseException.Type;

public class NhExceptionProvider {

    private static final String ERROR_CODE = "errorCode";

    public static NhBaseException nhExceptionBuilder(Exception e, Type type) {
        NhBaseException nhException = (NhBaseException)e;
        ThreadContext.put(ERROR_CODE, nhException.getStdErrorCode());
        StackTraceElement[] stack = e.getStackTrace();
        NhException.setMessageValue(((NhBaseException) e).getMessageValue());
        NhException.setMethodName(stack[0].getMethodName());
        NhException.setClassName(stack[0].getClassName());
        NhException.setFileName(stack[0].getFileName());
        NhException.setPgmLineNo(stack[0].getLineNumber());
        NhException.setStackTrace(stack);
        if (nhException.getErrMsgType() == null) {
            nhException.setErrMsgType(type);
        }
        return nhException;
    }

    public static NhBaseException nhExceptionBuilder(Exception e) {
        String errorCode = "FMW999";
        StackTraceElement target = Arrays.stream(e.getStackTrace())
            .filter(st -> st.getClassName().startsWith("nhis"))
            .findFirst()
            .orElse(e.getStackTrace()[0]);
        NhBaseException exception = new NhBaseException(errorCode);
        ThreadContext.put(ERROR_CODE, errorCode);
        exception.setMethodName(target.getMethodName());
        exception.setClassName(target.getClassName());
        exception.setErrMsgType(TYPE.RUNTIME);
        exception.setFileName(target.getFileName());
        exception.setPgmLineNo(target.getLineNumber());
        exception.setStackTrace(e.getStackTrace());
        String causeMessage = extractCauseMessage(e);
        // 원인 메시지를 추출하여 포함
        if (causeMessage != null && !causeMessage.isEmpty()) {
            exception.setAddMsgContents(causeMessage);
        }
    }

    /**
     * 원인 메시지를 재귀적으로 추출
     * 최대 3 단계까지 확인하여 가장 구체적인 메시지를 반환
     */
}

```

```

private static String extractCauseMessage(Exception e) {
    Throwable cause = e.getCause();
    int depth = 0;
    String lastMessage = null;
    while (cause != null && depth < 3) {
        String msg = cause.getMessage();
        if (msg != null && !msg.isEmpty()) {
            lastMessage = msg;
        }
        cause = cause.getCause();
        depth++;
    }
    // 마지막에 찾은 메시지 또는 예외 클래스명 포함
    if (lastMessage != null) {
        return lastMessage;
    }
    // 메시지 없으면 예외 클래스명 반환
    return e.getClass().getSimpleName();
}

package nhns.fw.commons.exception;

/**
 * <PRE>
 * 서비스 호출 시 예외
 * 서비스 간 호출 시 발생
 * </PRE>
 */
public class ServiceCommunicationException extends RuntimeException {

    public ServiceCommunicationException(String message, Throwable cause) {
        super(message, cause);
    }

    }

package nhns.fw.commons.exception;

/**
 * <PRE>
 * 서비스 호출 시 예외
 * 서비스 간 호출 시 발생
 * </PRE>
 */
public class ServiceHandlerNotFoundException extends RuntimeException {

    public ServiceHandlerNotFoundException(String message) {
        super(message);
    }

    public ServiceHandlerNotFoundException(String message, Throwable cause) {
        super(message, cause);
    }
}

```

```

    }
    public ServiceHandlerNotFound(Throwable cause) {
        super(cause);
    }
}

```

```

package nhnis.fw.commons.filter;

```

```

import java.io.IOException;
import java.util.Enumeration;
import java.util.Map;
import java.util.stream.Collectors;

```

```

import org.apache.logging.log4j.ThreadContext;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpConnector;
import org.springframework.stereotype.Component;

```

```

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.ObjectMapper;

```

```

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.FilterConfig;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;

```

```

import lombok.extern.slf4j.Slf4j;
import nhnis.fw.commons.context.ServiceContext;
import nhnis.fw.commons.dto.headerhdr_nhnis;
import nhnis.fw.commons.dto.header_sys_comm;
import nhnis.fw.commons.jwt.JwtProvider;

```

```

@Component
@Slf4j
public class DefaultFilter implements Filter {

```

```

    private final ClientHttpConnector connector;

```

```

    private static final String GUID
        = "guid";
    private static final String IP
        = "ip";
    private static final String USER_ID
        = "userid";
    private static final String SERVICE_ID
        = "serviceid";
    private static final String HDR_NHNIS
        = "hdr_nhnis";
    private static final String MULTIPART
        = "multipart/";

```

```

    @Value("${spring.profiles.active:local}")
    private String active;

```

```

    @Value("${spring.application.name}")
    private String applicationName;

```

```

    @Autowired

```

```

private JwtProvider jwtProvider;

DefaultFilter(ClientHttpConnector connector) {
    this.connector = connector;
}

@Override
public void init(FilterConfig filterConfig) {
    if (log.isDebugEnabled()) {
        log.debug("Application DefaultFilter is loaded");
    }
}

@Override
public void doFilter(ServletRequest servletRequest, ServletResponse servletResponse, FilterChain filterChain) throws IOException, ServletException {
    HttpServletRequest request = (HttpServletRequest) servletRequest;
    HttpServletResponse response = (HttpServletResponse) servletResponse;

    // ThreadContext 세팅
    String uri = request.getRequestURI();
    String serviceId = uri.contains("/") ? uri.substring(uri.lastIndexOf("/") + 1) : "UNKNOWN";
    ThreadContext.put(SERVICE_ID, serviceId);

    HttpHeaders requestHeaders = new HttpHeaders();
    Enumeration<String> names = request.getHeaderNames();
    while (names.hasMoreElements()) {
        String name = names.nextElement();
        requestHeaders.add(name, request.getHeader(name));
    }

    try {
        if (request.getContentType() != null &&
            request.getContentType().startsWith(MULTI_PART)) {
            String guid = request.getHeader("X-GUID");
            sys_comm sysComm = new sys_comm();
            sysComm.setStd_guid(guid);
            hdr_nhnis header = new hdr_nhnis();
            header.setSys_comm(sysComm);
            ThreadContext.put(guid, guid);
            ServiceContext serviceContext = new ServiceContext(applicationName,
                guid, active, requestHeaders, request, response, header);
            ServiceContextHolder.setInstance(serviceContext);
            filterChain.doFilter(request, servletResponse);
        } else {
            CachedBodyHttpServletRequest wrapper = new
                CachedBodyHttpServletRequest(request);

            /* SSO Check */
            if (active.toLowerCase().equals("local")) {
                String authorization =
                    request.getHeader(HttpHeaders.AUTHORIZATION);
                if (authorization == null || !authorization.startsWith("Bearer ")) {
                    response.sendError(HttpStatus.UNAUTHORIZED, "Access Token이 없습니다.");
                    return;
                }
                String token = authorization.substring(7);
            }
        }
    }
}

```

```

        if (jwtProvider.validate(token)) {
            response.sendError(HttpStatus.UNAUTHORIZED, "유효하지 않은 token입니다.");
            return;
        } else if (!jwtProvider.isAccessToken(token)) {
            response.sendError(HttpStatus.UNAUTHORIZED, "Access Token이 아닙니다.");
            return;
        }
        String ssold = jwtProvider.getssold(token);
        request.setAttribute("ssold", ssold);
    }
    String requestBody =
        wrapper.getReader().lines().collect(Collectors.joining(System.lineSeparator()));
    if (requestBody == null || requestBody.isEmpty()) {
        response.sendError(HttpStatus.BAD_REQUEST,
            "Request Body 입력 전문이 비어있습니다.");
        return;
    }
    ObjectMapper mapper = new ObjectMapper();
    mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
    Map<String, Object> rootMap = null;
    try {
        rootMap = mapper.readValue(requestBody, new
            TypeReference<Map<String, Object>>());
    } catch (JsonProcessingException e) {
        log.error("Request Body JSON 파싱 실패 : {}", e.getMessage(), e);
        response.sendError(HttpStatus.BAD_REQUEST,
            "잘못된 JSON 형식 입니다.");
        return;
    }
    Map<String, Object> headerMap = (Map<String, Object>)
        if (headerMap == null) {
            response.sendError(HttpStatus.BAD_REQUEST, "응답
                Header 정보가 없습니다.");
        }
        hdr_nhnis_header = convertMapToDto(mapper, headerMap,
            hdr_nhnis.class);
        String guid = header.getSys_comm().getStd_gbl_id();
        ThreadContext.put(Guid, header.getSys_comm().getStd_gbl_id());
        ThreadContext.put(IP, header.getSys_comm().getTrm_ipadr());
        ThreadContext.put(USER_ID, header.getSys_comm().getOptr_eno());
        ServiceContext serviceContext = new ServiceContext(applicationName,
            guid, active, requestHeaders, request, response, header);
        ServiceContextHolder.setInstance(serviceContext);
        // 서블릿 호출
        filterChain.doFilter(wrapper, servletResponse);
    } catch (Exception e) {
    }

```